PROJECT IDEATION · AI / ML FLAGSHIP PROJECT

PHYSICA

Figuring out WHY an assembly step went wrong — not just that it did

A camera watches a tabletop assembly task, reconstructs what happened, and ranks the most likely cause of a mistake — with the evidence attached.

Aarav | B.Tech IT, Maharaja Surajmal Institute of Technology, New Delhi

Ideation stage — concept and research plan, not a completed system

THE PROBLEM

Knowing something went wrong isn't the same as knowing why

When a trainee makes a mistake during a hands-on assembly task, we usually only see the final bad result. Figuring out when it went wrong — and what caused it — means re-watching footage by eye, which is slow and inconsistent. Most existing tools only detect THAT a mistake happened, not WHY.

WHAT MOST TOOLS SAY

“A mistake happened around frame 940.”

WHAT PHYSICA AIMS FOR

“Likely cause: connector placed reversed at step 2 — here's the evidence.”

A concrete use case, not every industry at once

WHO

New trainees learning a hands-on assembly task — like an electronics kit — and the instructors teaching them.

TODAY

The instructor watches live, rewatches footage afterward, or only checks the finished (often broken) result.

THE GAP

Manual review doesn't scale, and the final result alone can't show when or why the mistake happened.

Real cost: wasted kit parts, repeated training cycles, and instructor time spent re-explaining the same mistake instead of pointing to clear evidence. (No invented statistic here — this is described qualitatively.)

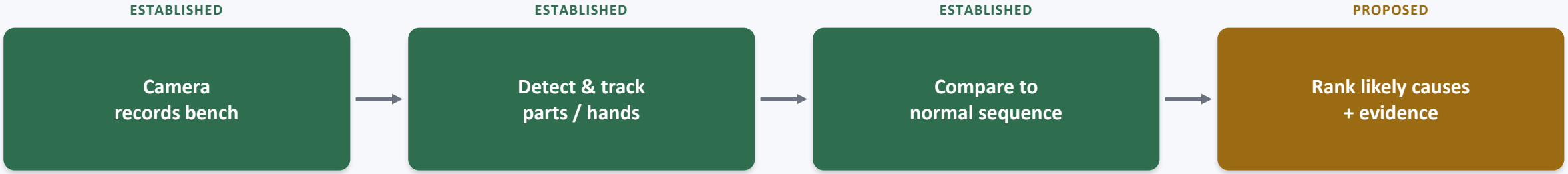
Active research area — the gap is specific, not total

Source	What it does	What it doesn't do
Assembly101 (CVPR 2022)	Dataset + benchmark for detecting assembly mistakes.	Labels that a mistake occurred — doesn't rank causes.
IndustReal (WACV 2024)	Detects missed or incorrect steps in an industrial-style task.	Flags the step, not the reason behind it.
BRIO-TA (ICIP 2022)	Toy-assembly dataset with 3 labeled error types.	Closest structurally, but still detection-only.
Fabrico / Spot.ai (commercial)	Video-AI tools selling root-cause analysis to factories.	Even their own materials call auto-root-cause “beta.”

PROPOSED SOLUTION

From camera input to a ranked, evidence-backed explanation

Example: pick connector → orient it → insert into slot → secure. If the connector goes in reversed at step 2, the final assembly fails inspection.



The first 3 steps directly extend techniques already proven in Assembly101 / IndustReal / BRIO-TA. The last step — ranking likely causes with evidence attached — is PHYSICA's own, unproven contribution.

A plain detector says: "connector, reversed, frame 322."

PHYSICA aims for: "likely cause: connector reversed at step 2; 2 other candidates ranked lower; evidence clip attached."

A specific claim, not a claim of total novelty

Ranked, evidence-linked causes — checked against a simple rule baseline — on one real tabletop task.

Realistic v1

- One bench, one kit
- Works on recorded clips, not live (at first)
- A rule baseline + one learned model
- No validated “what-if” simulator yet

Key challenges

- True causal attribution is hard — we rank candidates, not prove them
- Small dataset → results won't generalize beyond this pilot
- False alarms & detection delay need real testing
- Camera footage brings its own privacy/deployment questions

Sources & Next Steps

Key Sources

• Assembly101 (CVPR 2022)
https://openaccess.thecvf.com/content/CVPR2022/papers/Sener_Assembly101_A_Large-Scale_Multi-View_Video_Dataset_for_Understanding_Procedural_Activities_CVPR_2022_paper.pdf

• IndustReal (WACV 2024)
https://openaccess.thecvf.com/content/WACV2024/html/Schoonbeek_IndustReal_A_Dataset_for_Procedure_Step_Recognition_Handling_Execution_Errors_WACV_2024_paper.html

• BRIO-TA dataset (ICIP 2022) https://rc.signalprocessingsociety.org/conferences/icip-2022/spsicip22vid0818

• CLEVRER (NeurIPS 2019) https://arxiv.org/abs/1910.01442

• Fabrico (vendor, root-cause AI) https://www.fabrico.io/blog/ai-root-cause-analysis-software-guide-2026/

Next Steps

1. Pick one kit/task; record normal + seeded-mistake runs
2. Build the simple rule baseline first — it sets the bar
3. Fine-tune detection + step comparison on that pilot data
4. Test ranked-cause output against the known seeded cause

Open question:

Does a ranked, evidence-backed explanation actually help a trainee fix a mistake faster than a plain “mistake detected” label? That's the real hypothesis behind this whole project.